

# Jacobian Descent For Multi-Objective Optimization

Valérian Rey — Simplex Lab  
valerian.rey@gmail.com  
Joint work with Pierre Quinton

February 10, 2026



[github.com/TorchJD/torchjd](https://github.com/TorchJD/torchjd)



[torchjd.org](https://torchjd.org)



[arxiv.org/pdf/2406.16232](https://arxiv.org/pdf/2406.16232)

# Table of contents

## **Theory**

- Multi-objective optimization

- Pareto optimality

- First-order Taylor approximation

- Jacobian descent

- Unconflicting projection of gradients (UPGrad)

- Formal definition of UPGrad

## **Applications and experimentation**

- Multi-task learning

- Experimental results: per-sample JD

- Future directions

# Theory

# Multi-objective optimization

Define the partial order  $(\mathbb{R}^m, \preceq)$ , for  $\mathbf{u}, \mathbf{v} \in \mathbb{R}^m$ , as

$$\begin{aligned} & \mathbf{u} \preceq \mathbf{v} \\ \Leftrightarrow & \quad u_i \leq v_i \quad \forall i \in [m] \end{aligned}$$

# Multi-objective optimization

Define the partial order  $(\mathbb{R}^m, \preceq)$ , for  $\mathbf{u}, \mathbf{v} \in \mathbb{R}^m$ , as

$$\begin{aligned} \mathbf{u} &\preceq \mathbf{v} \\ \Leftrightarrow \quad u_i &\leq v_i \quad \forall i \in [m] \end{aligned}$$

Examples:

$$\blacktriangleright \begin{bmatrix} 2 \\ 1 \end{bmatrix} \preceq \begin{bmatrix} 2 \\ 2 \end{bmatrix}$$

# Multi-objective optimization

Define the partial order  $(\mathbb{R}^m, \preceq)$ , for  $\mathbf{u}, \mathbf{v} \in \mathbb{R}^m$ , as

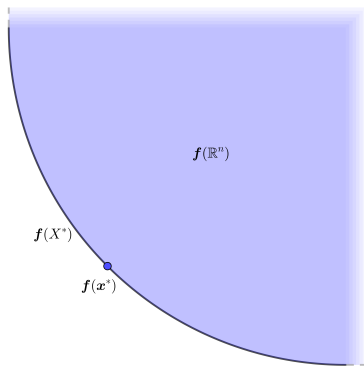
$$\begin{aligned} \mathbf{u} \preceq \mathbf{v} \\ \Leftrightarrow \quad u_i \leq v_i \quad \forall i \in [m] \end{aligned}$$

Examples:

▶  $\begin{bmatrix} 2 \\ 1 \end{bmatrix} \preceq \begin{bmatrix} 2 \\ 2 \end{bmatrix}$

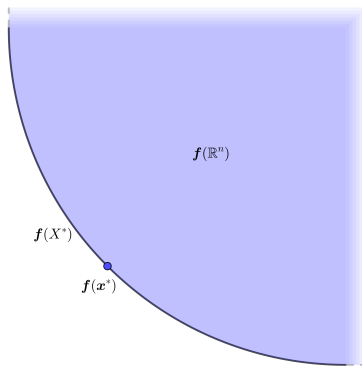
▶  $\begin{bmatrix} 2 \\ 1 \end{bmatrix}$  and  $\begin{bmatrix} 1 \\ 2 \end{bmatrix}$  are not comparable with this partial order.

# Pareto optimality



# Pareto optimality

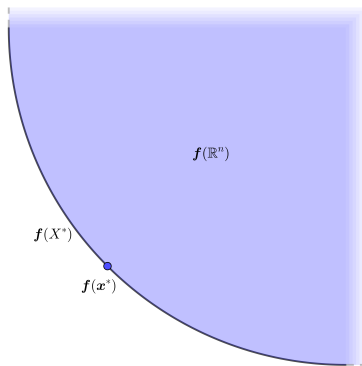
Pareto set:  $X^*$





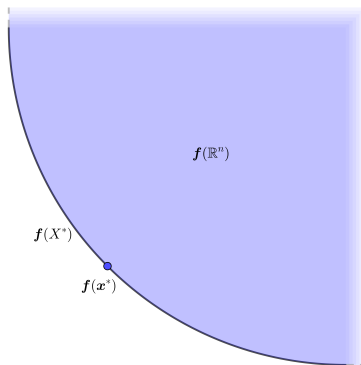
# Pareto optimality

Pareto set:  $X^*$   
Pareto Front:  $f(X^*)$



# Pareto optimality

Pareto set:  $X^*$   
Pareto Front:  $f(X^*)$



$$x^* \in X^* \quad \Leftrightarrow \quad \forall y \in \mathbb{R}^n, f(y) \preceq f(x^*) \rightarrow f(y) = f(x^*)$$

# First-order Taylor approximation

For any  $\mathbf{x} \in \mathbb{R}^n$ , the Jacobian of  $\mathbf{f}$  at  $\mathbf{x}$ , is

$$\mathcal{J}\mathbf{f}(\mathbf{x}) = \begin{bmatrix} \nabla f_1(\mathbf{x})^\top \\ \nabla f_2(\mathbf{x})^\top \\ \vdots \\ \nabla f_m(\mathbf{x})^\top \end{bmatrix} = \begin{bmatrix} \frac{\partial}{\partial x_1} f_1(\mathbf{x}) & \frac{\partial}{\partial x_2} f_1(\mathbf{x}) & \cdots & \frac{\partial}{\partial x_n} f_1(\mathbf{x}) \\ \frac{\partial}{\partial x_1} f_2(\mathbf{x}) & \frac{\partial}{\partial x_2} f_2(\mathbf{x}) & \cdots & \frac{\partial}{\partial x_n} f_2(\mathbf{x}) \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial}{\partial x_1} f_m(\mathbf{x}) & \frac{\partial}{\partial x_2} f_m(\mathbf{x}) & \cdots & \frac{\partial}{\partial x_n} f_m(\mathbf{x}) \end{bmatrix}$$

# First-order Taylor approximation

For any  $\mathbf{x} \in \mathbb{R}^n$ , the Jacobian of  $\mathbf{f}$  at  $\mathbf{x}$ , is

$$\mathcal{J}\mathbf{f}(\mathbf{x}) = \begin{bmatrix} \nabla f_1(\mathbf{x})^\top \\ \nabla f_2(\mathbf{x})^\top \\ \vdots \\ \nabla f_m(\mathbf{x})^\top \end{bmatrix} = \begin{bmatrix} \frac{\partial}{\partial x_1} f_1(\mathbf{x}) & \frac{\partial}{\partial x_2} f_1(\mathbf{x}) & \cdots & \frac{\partial}{\partial x_n} f_1(\mathbf{x}) \\ \frac{\partial}{\partial x_1} f_2(\mathbf{x}) & \frac{\partial}{\partial x_2} f_2(\mathbf{x}) & \cdots & \frac{\partial}{\partial x_n} f_2(\mathbf{x}) \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial}{\partial x_1} f_m(\mathbf{x}) & \frac{\partial}{\partial x_2} f_m(\mathbf{x}) & \cdots & \frac{\partial}{\partial x_n} f_m(\mathbf{x}) \end{bmatrix}$$

For a small enough update  $\mathbf{y} \in \mathbb{R}^n$ , Taylor's theorem states

$$\mathbf{f}(\mathbf{x} + \mathbf{y}) \approx \mathbf{f}(\mathbf{x}) + \mathcal{J}\mathbf{f}(\mathbf{x}) \cdot \mathbf{y}$$

# First-order Taylor approximation

For any  $\mathbf{x} \in \mathbb{R}^n$ , the Jacobian of  $\mathbf{f}$  at  $\mathbf{x}$ , is

$$\mathcal{J}\mathbf{f}(\mathbf{x}) = \begin{bmatrix} \nabla f_1(\mathbf{x})^\top \\ \nabla f_2(\mathbf{x})^\top \\ \vdots \\ \nabla f_m(\mathbf{x})^\top \end{bmatrix} = \begin{bmatrix} \frac{\partial}{\partial x_1} f_1(\mathbf{x}) & \frac{\partial}{\partial x_2} f_1(\mathbf{x}) & \cdots & \frac{\partial}{\partial x_n} f_1(\mathbf{x}) \\ \frac{\partial}{\partial x_1} f_2(\mathbf{x}) & \frac{\partial}{\partial x_2} f_2(\mathbf{x}) & \cdots & \frac{\partial}{\partial x_n} f_2(\mathbf{x}) \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial}{\partial x_1} f_m(\mathbf{x}) & \frac{\partial}{\partial x_2} f_m(\mathbf{x}) & \cdots & \frac{\partial}{\partial x_n} f_m(\mathbf{x}) \end{bmatrix}$$

For a small enough update  $\mathbf{y} \in \mathbb{R}^n$ , Taylor's theorem states

$$\mathbf{f}(\mathbf{x} + \mathbf{y}) \approx \mathbf{f}(\mathbf{x}) + \mathcal{J}\mathbf{f}(\mathbf{x}) \cdot \mathbf{y}$$

We select  $\mathbf{y} = -\eta \mathcal{A}(\mathcal{J}\mathbf{f}(\mathbf{x})) \in \mathbb{R}^n$ .

# Jacobian descent

The mapping  $\mathcal{A} : \mathbb{R}^{m \times n} \rightarrow \mathbb{R}^n$  is called an **aggregator**, it parameterizes **Jacobian descent**:

# Jacobian descent

The mapping  $\mathcal{A} : \mathbb{R}^{m \times n} \rightarrow \mathbb{R}^n$  is called an **aggregator**, it parameterizes **Jacobian descent**:

---

**Algorithm 1:** Jacobian descent with aggregator  $\mathcal{A}$

---

**Input:**  $\mathbf{x} \in \mathbb{R}^n$ ,  $0 < \eta$ ,  $T \in \mathbb{N}$ ,  $\mathcal{A} : \mathbb{R}^{m \times n} \rightarrow \mathbb{R}^n$

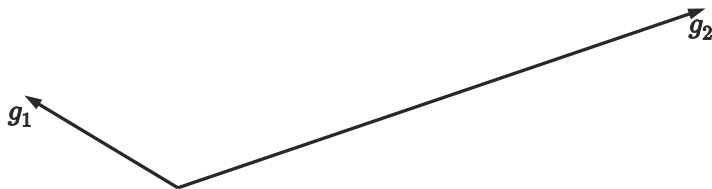
**for**  $t \leftarrow 1$  **to**  $T$  **do**

$\mathbf{x} \leftarrow \mathbf{x} - \eta \mathcal{A}(\mathcal{J}\mathbf{f}(\mathbf{x}))$

**Output:**  $\mathbf{x}$

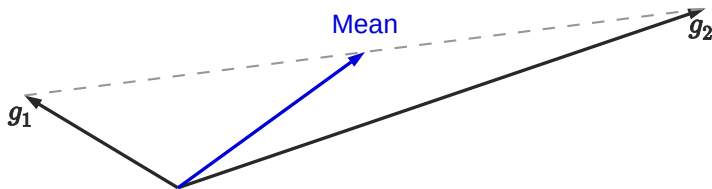
---

## Unconflicting projection of gradients (UPGrad)

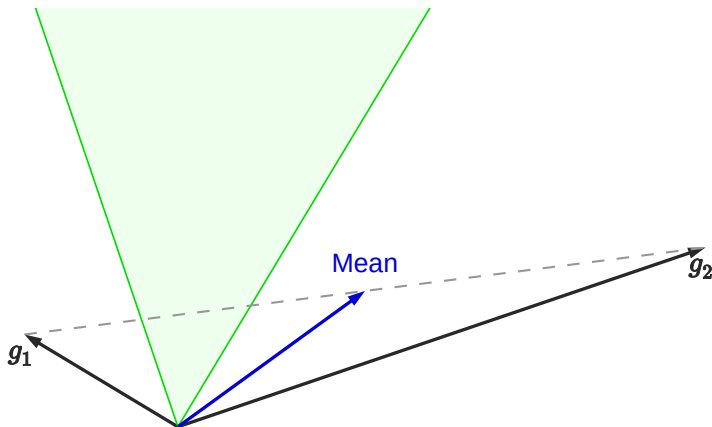




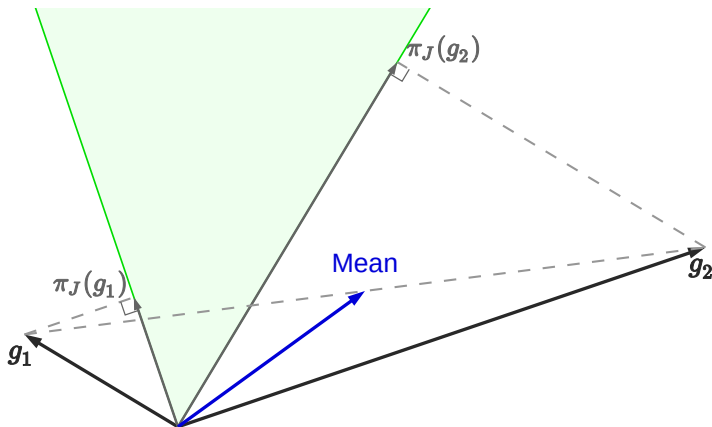
## Unconflicting projection of gradients (UPGrad)



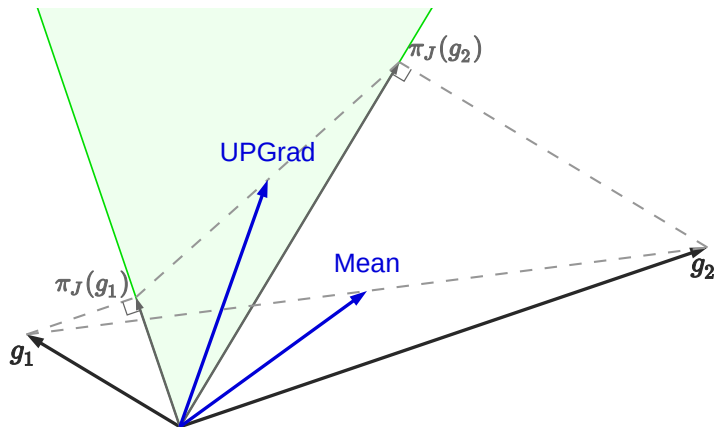
# Unconflicting projection of gradients (UPGrad)



# Unconflicting projection of gradients (UPGrad)



# Unconflicting projection of gradients (UPGrad)



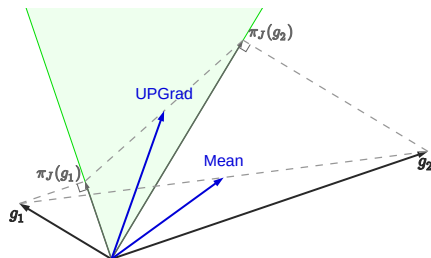
# Formal definition of UPGrad

Given  $J \in \mathbb{R}^{m \times n}$ , let  $C_J$  be the cone  $\{\mathbf{y} \in \mathbb{R}^n : \mathbf{0} \preceq J\mathbf{y}\}$ .  $\mathcal{A}$  is non-conflicting if for all  $J \in \mathbb{R}^{m \times n}$ ,  $\mathcal{A}(J) \in C_J$ .

For any  $\mathbf{x} \in \mathbb{R}^n$ , the projection of  $\mathbf{x}$  onto  $C_J$  is

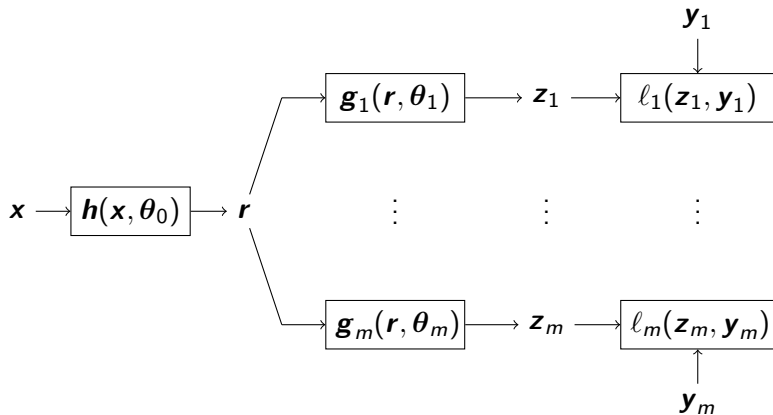
$$\pi_J(\mathbf{x}) = \arg \min_{\mathbf{y} \in C_J} \|\mathbf{x} - \mathbf{y}\|^2$$

$$\mathcal{A}_{\text{UPGrad}}(J) = \frac{1}{m} \sum_{i \in [m]} \pi_J(J^\top \mathbf{e}_i)$$

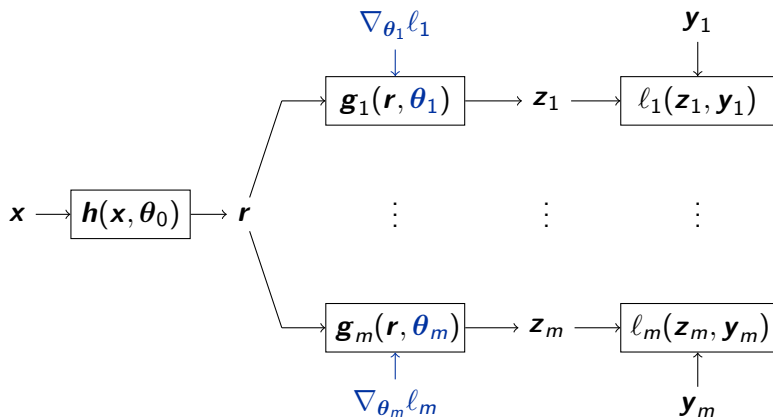


# Applications and experimentation

# Multi-task learning

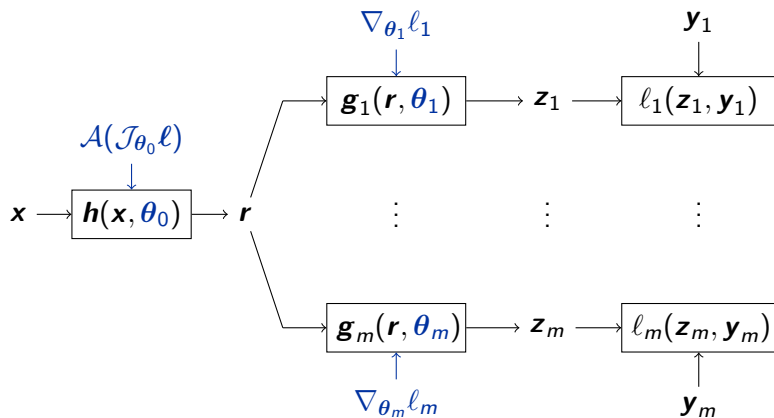


# Multi-task learning





# Multi-task learning



## Experimental results: per-sample JD

Each element of the batch has its own loss.

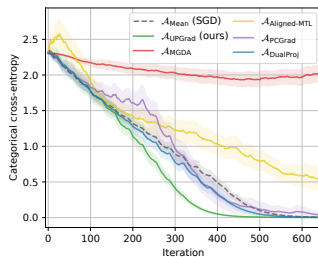
# Experimental results: per-sample JD

Each element of the batch has its own loss.

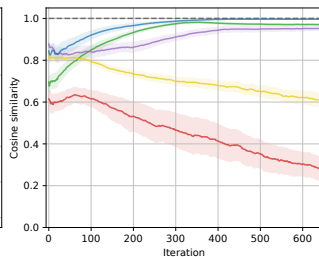
CIFAR-10



Training Loss



Update Similarity



# Future directions

## Implementation

- ▶ Make the `autogram` engine faster.

## Implementation

- ▶ Make the `autogram` engine faster.
- ▶ Make a better implementation of  $\mathcal{A}_{\text{UPGrad}}$ .

## Implementation

- ▶ Make the `autogram` engine faster.
- ▶ Make a better implementation of  $\mathcal{A}_{\text{UPGrad}}$ .
- ▶ Improve the interface of TorchJD.

## Implementation

- ▶ Make the `autogram` engine faster.
- ▶ Make a better implementation of  $\mathcal{A}_{\text{UPGrad}}$ .
- ▶ Improve the interface of TorchJD.

## Applications

- ▶ Are there multi-task learning problems with actual conflict?



## Implementation

- ▶ Make the `autogram` engine faster.
- ▶ Make a better implementation of  $\mathcal{A}_{\text{UPGrad}}$ .
- ▶ Improve the interface of TorchJD.

## Applications

- ▶ Are there multi-task learning problems with actual conflict?
- ▶ Could JD (per-sample) help training diffusion models?

## Implementation

- ▶ Make the `autogram` engine faster.
- ▶ Make a better implementation of  $\mathcal{A}_{\text{UPGrad}}$ .
- ▶ Improve the interface of TorchJD.

## Applications

- ▶ Are there multi-task learning problems with actual conflict?
- ▶ Could JD (per-sample) help training diffusion models?
- ▶ Could JD (per-depth) help training recurrent models?

## Implementation

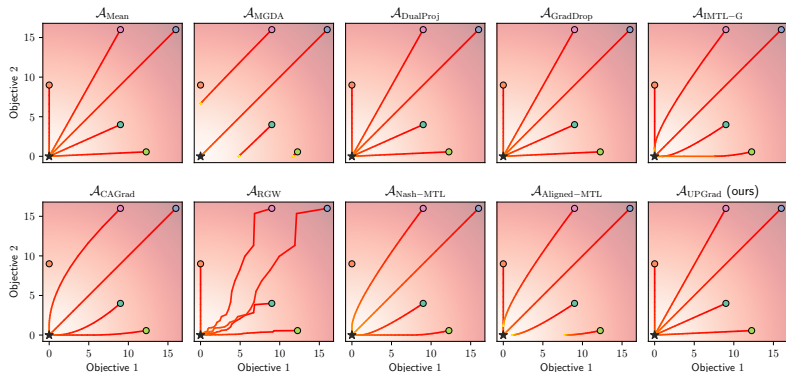
- ▶ Make the **autogram** engine faster.
- ▶ Make a better implementation of  $\mathcal{A}_{\text{UPGrad}}$ .
- ▶ Improve the interface of TorchJD.

## Applications

- ▶ Are there multi-task learning problems with actual conflict?
- ▶ Could JD (per-sample) help training diffusion models?
- ▶ Could JD (per-depth) help training recurrent models?
- ▶ Find new applications.

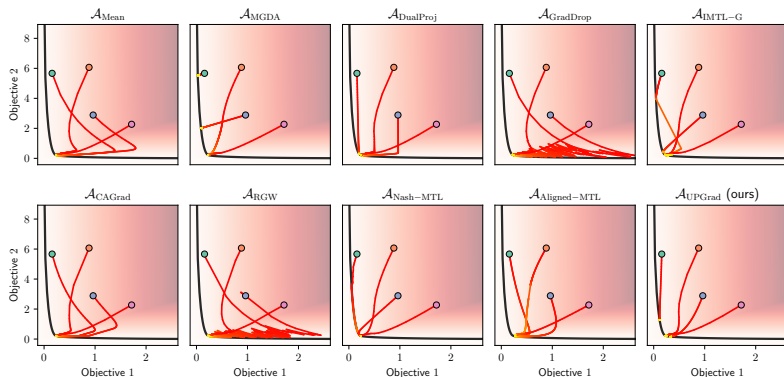
## Bonus: Element-wise quadratic function trajectories

Minimize  $\mathbf{f} : \begin{bmatrix} x \\ y \end{bmatrix} \mapsto \begin{bmatrix} x^2 \\ y^2 \end{bmatrix}$ . It's Jacobian at  $\begin{bmatrix} x \\ y \end{bmatrix}$  is  $\begin{bmatrix} 2x & 0 \\ 0 & 2y \end{bmatrix}$ .



## Bonus: Convex quadratic form trajectories

Some convex quadratic function  $\mathbf{f} : \mathbb{R}^2 \rightarrow \mathbb{R}^2$ , with highly conflicting gradients.



## Bonus: Gramian-based JD

$\mathcal{A}_{\text{UPGrad}}(\mathcal{J}\mathbf{f}(\mathbf{x}))$  is a linear combination of the rows of  $\mathcal{J}\mathbf{f}(\mathbf{x})$ .

## Bonus: Gramian-based JD

$\mathcal{A}_{\text{UPGrad}}(\mathcal{J}\mathbf{f}(\mathbf{x}))$  is a linear combination of the rows of  $\mathcal{J}\mathbf{f}(\mathbf{x})$ .

The weights only depend on the (small) Gramian  
 $\mathcal{G}\mathbf{f}(\mathbf{x}) = \mathcal{J}\mathbf{f}(\mathbf{x}) \cdot \mathcal{J}\mathbf{f}(\mathbf{x})^\top$ .

## Bonus: Gramian-based JD

$\mathcal{A}_{\text{UPGrad}}(\mathcal{J}\mathbf{f}(\mathbf{x}))$  is a linear combination of the rows of  $\mathcal{J}\mathbf{f}(\mathbf{x})$ .

The weights only depend on the (small) Gramian  
 $\mathcal{G}\mathbf{f}(\mathbf{x}) = \mathcal{J}\mathbf{f}(\mathbf{x}) \cdot \mathcal{J}\mathbf{f}(\mathbf{x})^\top$ .

We can compute  $\mathcal{G}\mathbf{f}(\mathbf{x})$  directly, extract the weights, combine the losses, and do a step of gradient descent.



## Bonus: Gramian-based JD

$\mathcal{A}_{\text{UPGrad}}(\mathcal{J}\mathbf{f}(\mathbf{x}))$  is a linear combination of the rows of  $\mathcal{J}\mathbf{f}(\mathbf{x})$ .

The weights only depend on the (small) Gramian  
 $\mathcal{G}\mathbf{f}(\mathbf{x}) = \mathcal{J}\mathbf{f}(\mathbf{x}) \cdot \mathcal{J}\mathbf{f}(\mathbf{x})^\top$ .

We can compute  $\mathcal{G}\mathbf{f}(\mathbf{x})$  directly, extract the weights, combine the losses, and do a step of gradient descent.

We have developed the `autogram` engine to compute  $\mathcal{G}\mathbf{f}(\mathbf{x})$  efficiently.

## Bonus: Gramian-based JD

$\mathcal{A}_{\text{UPGrad}}(\mathcal{J}\mathbf{f}(\mathbf{x}))$  is a linear combination of the rows of  $\mathcal{J}\mathbf{f}(\mathbf{x})$ .

The weights only depend on the (small) Gramian  
 $\mathcal{G}\mathbf{f}(\mathbf{x}) = \mathcal{J}\mathbf{f}(\mathbf{x}) \cdot \mathcal{J}\mathbf{f}(\mathbf{x})^\top$ .

We can compute  $\mathcal{G}\mathbf{f}(\mathbf{x})$  directly, extract the weights, combine the losses, and do a step of gradient descent.

We have developed the `autogram` engine to compute  $\mathcal{G}\mathbf{f}(\mathbf{x})$  efficiently.

This saves a lot of memory, so it's often much faster.

## Bonus: Gramian-based JD

$\mathcal{A}_{\text{UPGrad}}(\mathcal{J}\mathbf{f}(\mathbf{x}))$  is a linear combination of the rows of  $\mathcal{J}\mathbf{f}(\mathbf{x})$ .

The weights only depend on the (small) Gramian  
 $\mathcal{G}\mathbf{f}(\mathbf{x}) = \mathcal{J}\mathbf{f}(\mathbf{x}) \cdot \mathcal{J}\mathbf{f}(\mathbf{x})^\top$ .

We can compute  $\mathcal{G}\mathbf{f}(\mathbf{x})$  directly, extract the weights, combine the losses, and do a step of gradient descent.

We have developed the `autogram` engine to compute  $\mathcal{G}\mathbf{f}(\mathbf{x})$  efficiently.

This saves a lot of memory, so it's often much faster.

Applicable to most other aggregators from the literature.